



SERVIÇO PÚBLICO FEDERAL · MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE VIÇOSA · UFV
CAMPUS FLORESTAL

Trabalho Prático 3 - Sistemas Distribuídos

HENRIQUE ALVES CAMPOS [EF04231]

HENRIQUE DE SOUZA CAMPOS [EF04673]

HERON FILLIPE SILVEIRA SANTOS [EF04211]

Sumário

Sumário.....	2
1. Introdução.....	3
2. Alterações de Projeto.....	3
2.1. Reformulação do Tema.....	3
2.2. Substituição da Interface Gráfica.....	3
2.3. Remoção da Operação de Edição.....	4
2.4. Adaptação do Modelo de Dados.....	4
3. Arquitetura e Componentes do Sistema.....	4
3.1. Servidor.....	4
3.2. Cliente.....	6
4. Protocolo de Mensagens.....	7
4.1. Mensagens do Cliente para o Servidor.....	7
4.2. Mensagens do Servidor para o Cliente.....	9
4.3. Fluxo de Mensagens.....	10
5. Uso de Threads e Sincronização.....	13
5.1. Threads no Servidor.....	13
5.2. Sincronização no Servidor.....	14
5.3. Thread no Cliente.....	14
5.4. Comunicação entre a Thread de Rede e a Interface Gráfica.....	15
6. Conclusão.....	15
Referências.....	16

1. Introdução

Esta terceira etapa do trabalho prático dá continuidade ao desenvolvimento do sistema distribuído iniciado nas partes anteriores. Enquanto a Parte 1 estabeleceu os requisitos funcionais e os casos de uso do sistema, e a Parte 2 definiu sua modelagem arquitetural e os modelos fundamentais, esta fase concentra-se na implementação do sistema utilizando a API de Sockets.

Ao longo do desenvolvimento, o sistema passou por uma reformulação conceitual descrita na Seção 2: o tema original do Papel-it foi substituído pelo GridCity, um ambiente colaborativo de construção urbana no qual múltiplos usuários interagem sobre uma grade 7x7, alocando e removendo estruturas urbanas. Essa mudança não alterou os fundamentos arquiteturais definidos na Parte 2, tendo seu impacto concentrado na camada de dados e na interface gráfica.

A implementação foi realizada integralmente em Python, utilizando sockets TCP para comunicação entre processos, PyQt6 para a interface gráfica, SQLite para persistência do estado e threads para gerenciamento de concorrência. O código-fonte do projeto está disponível no repositório público: <https://github.com/alveshenrique/CCF-355-SistemasDistribuidos>

2. Alterações de Projeto

2.1 Reformulação do Tema

O sistema originalmente concebido como Papel-it, um mural digital de post-its compartilhado, foi reformulado para o GridCity, um ambiente colaborativo de construção urbana. Nesse novo modelo, múltiplos usuários interagem sobre uma grade 7x7, alocando e removendo estruturas urbanas pré-definidas, como casas, estradas, hospitais e outros elementos urbanos.

A mudança de tema não alterou os fundamentos arquiteturais definidos na Parte 2. O modelo físico, a arquitetura cliente-servidor e o paradigma de comunicação por troca de mensagens via TCP permaneceram inalterados. O impacto foi concentrado na camada de dados e na interface gráfica.

2.2 Substituição da Interface Gráfica: Tkinter para PyQt6

A interface gráfica originalmente planejada com Tkinter foi substituída pelo PyQt6. A decisão se deu pela exigência da especificação da Parte 3, que tornou o uso do PyQt6 obrigatório. Além disso, o PyQt6 oferece um mecanismo nativo de comunicação entre threads por meio de sinais e slots, o que simplificou a integração entre a thread de rede e a interface gráfica, evitando condições de corrida na atualização da UI.

2.3 Remoção da Operação de Edição

O caso de uso UC04 — Editar Papel-it, previsto na Parte 1, não foi implementado no GridCity. Essa decisão decorreu de uma análise do novo modelo de domínio: no Papel-it, a edição fazia sentido pois o post-it possuía conteúdo textual livre que poderia ser alterado. No GridCity, cada estrutura possui apenas um tipo fixo, sem conteúdo parcialmente modificável.

Dessa forma, a operação de edição foi naturalmente substituída pelo fluxo de remoção seguida de reinserção: o usuário remove a estrutura existente e aloca uma nova estrutura do tipo desejado na mesma posição. Esse fluxo atende ao mesmo objetivo sem necessidade de uma operação dedicada, simplificando o protocolo de comunicação. O fluxo passa a ser coberto pelos casos de uso UC03 e UC05 em sequência.

2.4 Adaptação do Modelo de Dados

No Papel-it, cada elemento da matriz armazenava um conteúdo textual livre associado a um autor e horário. No GridCity, esse modelo foi adaptado: cada célula da grade armazena uma estrutura composta por tipo, autor e horário, sem campo de conteúdo livre. Essa simplificação reflete diretamente o novo domínio do sistema e está implementada na classe Structure em `src/server/models/structure.py`.

3. Arquitetura e Componentes do Sistema

3.1 Servidor

O servidor é composto por três módulos principais e um modelo de dados.

server.py

Ponto de entrada do servidor. Responsável por inicializar o socket TCP, vinculá-lo ao endereço e porta definidos (0.0.0.0:5000) e aguardar conexões em loop contínuo.

Para cada novo cliente aceito, instancia um `ClientHandler` e o executa em uma thread dedicada, permitindo que múltiplos clientes sejam atendidos de forma simultânea e independente.

state_manager.py

Responsável pelo gerenciamento do estado global da grade 7x7 e pela persistência dos dados em banco SQLite. Ao ser instanciado, cria a tabela de estruturas caso não exista e carrega o estado persistido para a memória. O `threading.Lock` é utilizado para proteger o acesso à grade em memória, garantindo que acessos concorrentes de múltiplas threads não resultem em inconsistências. As operações de persistência no banco são realizadas fora do lock, após a atualização da grade. Expõe três métodos principais:

- `place_structure(x, y, structure_type, author)` — valida se a posição está livre, insere a estrutura na grade em memória e persiste no banco
- `remove_structure(x, y)` — valida se a posição está ocupada, remove da grade em memória e do banco
- `get_state()` — retorna a grade serializada como lista de listas de dicionários

client_handler.py

Executado em uma thread por cliente conectado. Responsável por receber e processar as mensagens enviadas pelo cliente, encaminhar as operações ao `StateManager` e propagar as respostas. Seu fluxo de execução é:

1. Recebe e processa a mensagem de JOIN de forma isolada, antes do loop principal, registrando o nome do usuário
2. Adiciona o cliente à lista compartilhada e notifica todos com a lista atualizada de usuários
3. Envia o estado atual da grade ao cliente recém-conectado
4. Entra em loop de recebimento de mensagens, processando exclusivamente PLACE e REMOVE
5. Após cada operação bem-sucedida, faz broadcast do novo estado para todos os clientes
6. Ao encerrar a conexão, remove o cliente da lista e notifica os demais com a lista atualizada. As estruturas alocadas pelo usuário permanecem na grade após sua desconexão

models/structure.py

Define a classe `Structure`, que representa uma estrutura alocada em uma célula da grade. Armazena `structure_type`, `author` e `timestamp`, e fornece o método `to_dict()` para serialização em JSON durante o broadcast do estado.

3.2 Cliente

O cliente é composto por quatro módulos com responsabilidades bem separadas.

client.py

Ponto de entrada do cliente. Sua função é exclusivamente inicializar a aplicação e invocar a função `main()` definida em `ui.py`. Essa separação é uma decisão de design consciente: isolar o ponto de entrada da lógica de interface facilita a manutenção e permite que o módulo de UI seja importado e testado de forma independente.

protocol.py

Responsável pela construção das mensagens enviadas ao servidor. Define três funções que retornam dicionários no formato esperado pelo protocolo:

- `create_join(username)` — mensagem de identificação do usuário
- `create_place(x, y, structure_type, author)` — requisição de alocação de estrutura
- `create_remove(x, y)` — requisição de remoção de estrutura

network.py

Responsável pela comunicação TCP com o servidor. Composto por duas classes:

- `NetworkSignals` — define os sinais Qt utilizados para comunicação segura entre a thread de rede e a interface gráfica: `state_updated`, `users_updated` e `response_received`
- `NetworkClient` — gerencia a conexão TCP, o envio de mensagens e o recebimento assíncrono. Ao conectar, inicia uma thread daemon que escuta o socket continuamente. Utiliza um buffer acumulativo com `raw_decode()` para

tratar corretamente mensagens fragmentadas pelo TCP, emitindo os sinais correspondentes para cada tipo de mensagem recebida

Vale destacar que o PyQt6 impõe restrições ao acesso à interface gráfica fora da thread principal. O uso de sinais e slots como mecanismo de comunicação entre a thread de rede e a UI foi a estratégia adotada para garantir que todas as atualizações visuais ocorram de forma segura, evitando condições de corrida.

ui.py

Responsável pela interface gráfica, construída integralmente com PyQt6. Composto por três componentes principais:

- `LoginDialog` — diálogo de autenticação onde o usuário informa seu nome antes de entrar na cidade
- `CityGrid` — widget que renderiza a grade 7x7 via `QPainter`, com suporte a hover e cliques com os botões esquerdo e direito do mouse
- `MainWindow` — janela principal que integra o grid, o painel lateral com seletor de estruturas, lista de usuários online e log de atividade recente. Instancia o `NetworkClient`, conecta os sinais de rede aos slots de atualização da interface e coordena toda a interação do usuário com o sistema

4. Protocolo de Mensagens

Toda a comunicação entre cliente e servidor é realizada via TCP, com mensagens no formato JSON codificadas em UTF-8. A escolha do JSON como formato de serialização se deu pela sua legibilidade, pela facilidade de depuração durante o desenvolvimento e pela disponibilidade nativa na biblioteca padrão do Python. O campo `type` está presente em todas as mensagens e é o primeiro campo verificado pelo receptor para determinar como processar o conteúdo recebido.

Como o TCP não garante que cada `recv` corresponda a exatamente uma mensagem, o cliente utiliza um buffer acumulativo em conjunto com `json.JSONDecoder().raw_decode()` para extrair corretamente um objeto JSON por vez do fluxo de bytes recebido, tratando de forma transparente eventuais fragmentações impostas pelo protocolo.

4.1 Mensagens do Cliente para o Servidor

O cliente é responsável por iniciar todas as operações do sistema. As mensagens enviadas ao servidor são sempre requisições de ação, e o servidor nunca envia mensagens sem que um evento as origine.

JOIN

Enviada imediatamente após a conexão TCP ser estabelecida. É a primeira mensagem do protocolo e identifica o usuário perante o servidor. O servidor a processa de forma isolada, antes do loop principal de mensagens, e utiliza o nome recebido em todos os broadcasts subsequentes. Caso o JOIN não seja recebido corretamente, o servidor utiliza o endereço IP do cliente como identificador de fallback.

```
JSON
{
  "type": "JOIN",
  "username": "Henrique"
}
```

PLACE

Enviada quando o usuário clica com o botão esquerdo em uma célula vazia da grade, solicitando a alocação de uma estrutura. Ao receber essa mensagem, o servidor valida se a posição está livre, registra a estrutura associando-a ao autor e ao horário da operação, persiste no banco e responde ao cliente com uma mensagem RESPONSE. Em caso de sucesso, faz broadcast do novo estado para todos os clientes conectados.

```
JSON
{
  "type": "PLACE",
  "x": 3,
  "y": 5,
  "structure_type": "Casa",
  "author": "Henrique"
}
```

REMOVE

Enviada quando o usuário clica com o botão direito em uma célula ocupada da grade, solicitando a remoção da estrutura existente. O servidor valida se a posição está de fato ocupada antes de executar a remoção. Em caso de sucesso, remove a estrutura da grade em memória e do banco, e faz broadcast do novo estado.

JSON

```
{
  "type": "REMOVE",
  "x": 3,
  "y": 5
}
```

4.2 Mensagens do Servidor para o Cliente

O servidor nunca inicia comunicação de forma espontânea. Todas as mensagens enviadas pelo servidor são consequência de um evento: uma requisição de um cliente ou uma mudança no estado do sistema.

RESPONSE

Enviada exclusivamente ao cliente que originou a operação, informando o resultado de um PLACE ou REMOVE. O campo success indica se a operação foi executada com sucesso. Em caso de falha, o campo message descreve o motivo, por exemplo "Posição já ocupada" ou "Posição já está vazia". O cliente utiliza essa mensagem para exibir feedback imediato ao usuário no log de atividade da interface.

JSON

```
{
  "type": "RESPONSE",
  "success": true,
  "message": "Estrutura alocada",
  "action": "PLACE",
  "x": 3,
  "y": 5,
  "structure_type": "Casa"
}
```

UPDATE

É a mensagem mais importante do protocolo, pois é ela que garante a consistência do estado da grade entre todos os clientes conectados. É enviada em duas situações distintas: individualmente ao cliente no momento de sua conexão, para sincronizar o estado inicial, e em broadcast a todos os clientes após cada operação bem-sucedida de PLACE ou REMOVE. O estado é sempre enviado completo, ou seja, a grade inteira 7x7, evitando a necessidade de lógica de aplicação de diffs no cliente.

```
JSON
{
  "type": "UPDATE",
  "state": [
    [null, {"type": "Casa", "author": "Henrique", "time": "14:32:10"}, null],
    ...
  ]
}
```

Cada célula da matriz é `null` se vazia, ou um objeto com `type`, `author` e `time` se ocupada.

USERS

Enviada em broadcast a todos os clientes conectados sempre que um usuário entra ou sai do sistema. O cliente utiliza essa mensagem para atualizar o painel de usuários online exibido na interface. O campo `count` indica o número atual de usuários conectados e `usernames` contém a lista completa de identificadores.

```
JSON
{
  "type": "USERS",
  "count": 2,
  "usernames": ["Henrique", "Heron"]
}
```

4.3 Fluxo de Mensagens

A seguir são apresentados os fluxos de troca de mensagens para os principais cenários do sistema.

Conexão de um novo cliente

Ao estabelecer a conexão TCP, o cliente envia imediatamente o JOIN. O servidor responde com um broadcast de USERS para todos os clientes conectados, incluindo o novo, e em seguida envia o UPDATE com o estado atual da grade exclusivamente ao cliente recém-conectado.

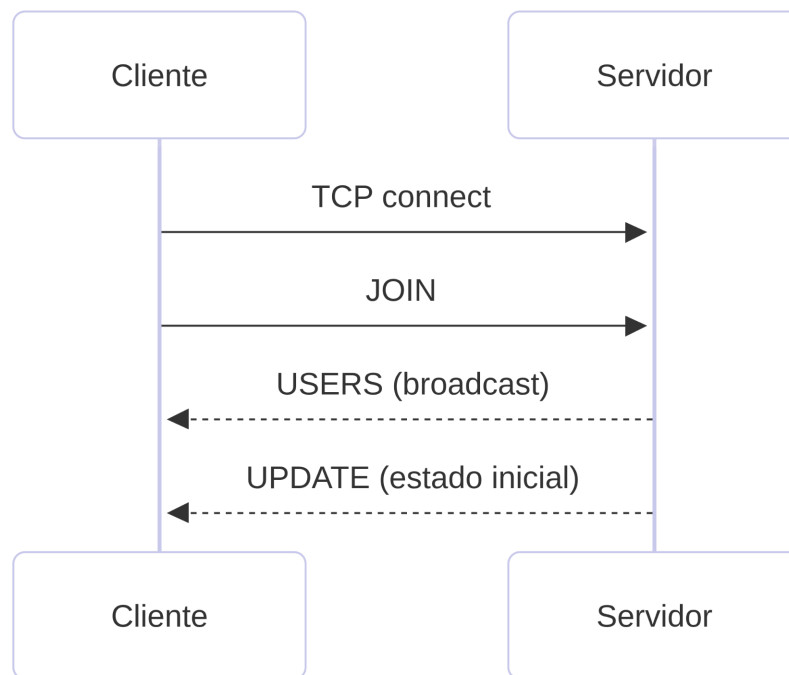


Figura 1 - Diagrama de conexão

Alocação de estrutura com sucesso

O cliente envia PLACE com a posição e o tipo de estrutura desejados. O servidor valida, executa e responde com RESPONSE ao cliente originador. Em seguida faz broadcast do UPDATE com o novo estado para todos os clientes.

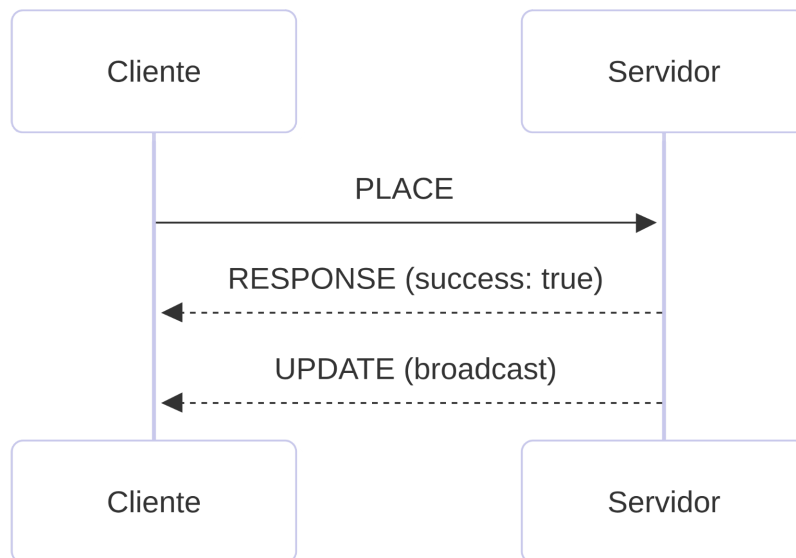


Figura 2 - Diagrama Alocação com sucesso

Alocação em posição ocupada

Quando a posição solicitada já está ocupada, o servidor responde apenas com RESPONSE indicando falha. Nenhum broadcast é realizado pois o estado da grade não foi alterado.

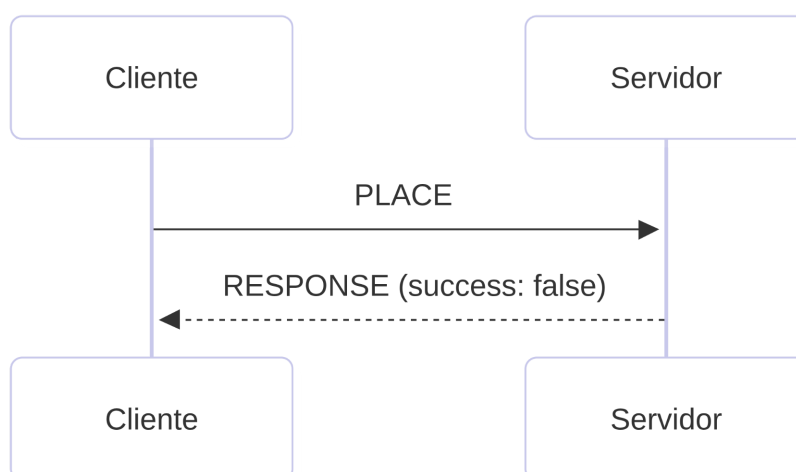


Figura 3 - Diagrama Alocação em posição ocupada

Remoção de estrutura com sucesso

O cliente envia REMOVE com a posição da estrutura a ser removida. O servidor valida a existência da estrutura, executa a remoção e responde com RESPONSE ao cliente originador, seguido de broadcast do UPDATE para todos.

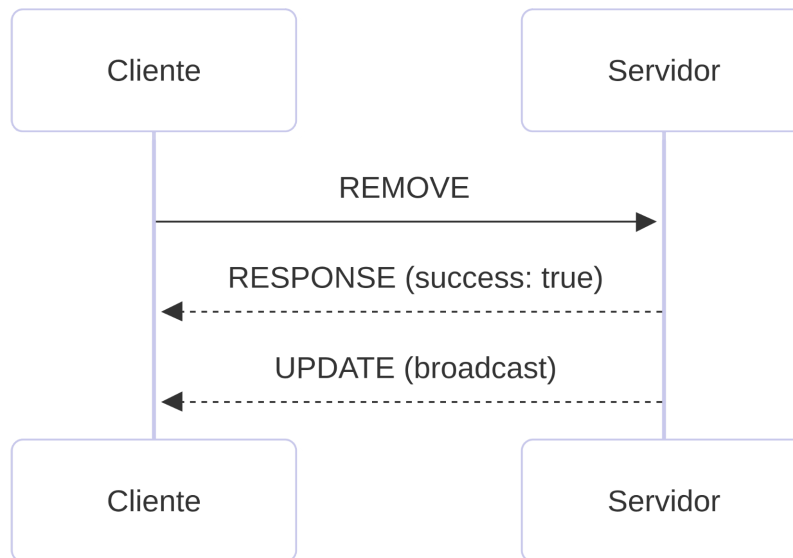


Figura 4 - Diagrama Remoção com sucesso

Desconexão

Ao encerrar a conexão, o servidor remove o cliente da lista de conectados e faz broadcast de USERS com a lista atualizada. As estruturas alocadas pelo usuário permanecem na grade após sua desconexão.

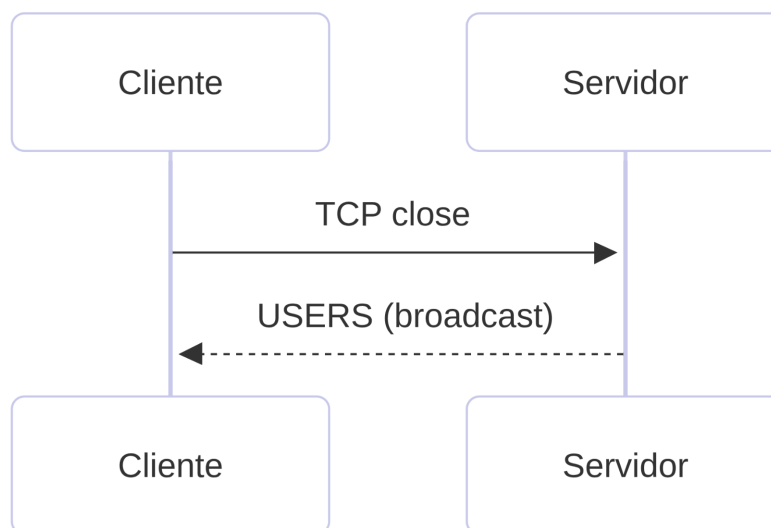


Figura 5 - Diagrama Desconexão

5. Uso de Threads e Sincronização

O uso correto de threads é um dos aspectos centrais da implementação do GridCity, sendo necessário tanto no servidor, para atender múltiplos clientes de forma simultânea, quanto no cliente, para manter a interface gráfica responsiva durante a comunicação com o servidor. Esta seção descreve onde, como e por que as threads foram utilizadas, além dos mecanismos de sincronização adotados para garantir a consistência do estado compartilhado.

5.1 Threads no Servidor

O servidor utiliza duas categorias de threads. A thread principal executa o loop de aceitação de conexões TCP em `server.py`, aguardando novas conexões com `server.accept()` e, para cada cliente aceito, instanciando um `ClientHandler` e delegando seu atendimento a uma nova thread. Essa separação garante que a chegada de um novo cliente nunca bloqueie o atendimento dos demais, pois a thread principal retorna imediatamente ao loop de espera após delegar.

Para cada cliente conectado, uma thread dedicada executa o método `handle()` do `ClientHandler`, sendo responsável por todo o ciclo de vida daquela conexão: receber o JOIN, registrar o cliente, enviar o estado inicial, manter o loop de processamento de mensagens e detectar a desconexão. Todas as threads de atendimento são criadas com `daemon=True`, garantindo que sejam encerradas automaticamente quando o processo principal for finalizado.

5.2 Sincronização no Servidor

O principal desafio de concorrência no servidor é o acesso simultâneo à grade compartilhada por múltiplas threads. Sem controle, duas threads poderiam verificar simultaneamente que uma posição está livre e ambas tentariam alocar uma estrutura no mesmo lugar, corrompendo o estado do sistema.

Para evitar esse problema, o `StateManager` utiliza um `threading.Lock` que protege todas as operações de leitura e escrita na grade em memória. O uso do bloco `with self.lock` garante que apenas uma thread por vez execute a verificação e a modificação de uma célula, tornando a operação atômica. A escrita no banco de dados é realizada fora do lock intencionalmente, pois o SQLite possui seu próprio

mecanismo de controle de acesso concorrente, e manter o lock por menos tempo reduz a contenção entre as threads.

A lista `clients`, compartilhada entre todas as threads para broadcast, não possui um Lock explícito. Para o escopo do trabalho isso não causou inconsistências observadas, mas em um cenário de alta concorrência essa seria uma área a ser reforçada com um mecanismo de proteção dedicado.

5.3 Thread no Cliente

No lado do cliente, o maior desafio é receber mensagens do servidor de forma contínua sem bloquear a interface gráfica. O PyQt6 executa sua interface na thread principal, e qualquer operação bloqueante nessa thread congela a janela e impede a interação do usuário.

Para resolver esse problema, o `NetworkClient` inicia uma thread daemon exclusivamente responsável por escutar o socket. Essa thread executa um loop contínuo de recebimento, acumulando os dados em um buffer e extraíndo mensagens JSON completas com `raw_decode()`. O uso do buffer acumulativo é necessário porque o TCP não garante que uma mensagem JSON chegue completa em um único `recv()`, podendo ser fragmentada em múltiplos pacotes ou múltiplas mensagens podem chegar concatenadas em um único pacote.

5.4 Comunicação entre a Thread de Rede e a Interface Gráfica

Atualizar diretamente elementos da interface a partir de uma thread secundária é inseguro no PyQt6 e pode causar comportamentos imprevisíveis. Para garantir a comunicação segura entre a thread de rede e a thread da UI, o sistema utiliza o mecanismo de sinais e slots do Qt por meio da classe `NetworkSignals`, que define três sinais: `state_updated`, `users_updated` e `response_received`.

Quando a thread de rede recebe uma mensagem, ela emite o sinal correspondente. O Qt garante que os slots conectados a esses sinais sejam executados na thread principal da UI, de forma segura e sem condições de corrida, independente de qual thread emitiu o sinal. Esse mecanismo elimina a necessidade de locks explícitos no lado do cliente e é a abordagem recomendada pelo próprio framework para integração entre threads e interface gráfica.

6. Conclusão

A implementação do GridCity utilizando a API de Sockets representou um contato direto com os desafios inerentes ao desenvolvimento de sistemas distribuídos sem o auxílio de middlewares. Esta seção relata as principais dificuldades encontradas ao longo do processo.

A primeira delas foi a necessidade de projetar um protocolo de comunicação do zero. Sem um middleware, toda a estrutura das mensagens, os campos obrigatórios e as respostas esperadas precisaram ser definidos e implementados manualmente, tanto no servidor quanto no cliente.

A segunda dificuldade foi a fragmentação de mensagens imposta pelo TCP. Por ser um protocolo de fluxo de bytes, um único `recv()` pode retornar uma mensagem incompleta ou múltiplas mensagens concatenadas. Sem o tratamento adequado, isso causaria erros silenciosos de parsing. A solução foi a implementação de um buffer acumulativo com `raw_decode()`, que permitiu extrair mensagens JSON corretamente independente de como os bytes foram entregues.

Por fim, a integração entre a thread de rede e a interface gráfica exigiu atenção especial, dado que o PyQt6 restringe a manipulação da UI à thread principal. O mecanismo de sinais e slots do Qt resolveu esse problema de forma elegante e sem necessidade de locks explícitos no cliente.

De modo geral, a experiência evidenciou com clareza o nível de complexidade que middlewares abstraem do desenvolvedor. As dificuldades encontradas, embora desafiadoras, foram superadas com soluções bem fundamentadas, e a comparação com a abordagem via RMI, prevista para a Parte 4, deverá tornar ainda mais evidente os benefícios que middlewares oferecem em sistemas de maior escala.

Referências

- [1] TANENBAUM, Andrew S.; VAN STEEN, Maarten. Sistemas Distribuídos: Princípios e Paradigmas. 2. ed. São Paulo: Pearson, 2007.
- [2] COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. Distributed Systems: Concepts and Design. 5. ed. Boston: Addison-Wesley, 2011.

[3] PYTHON SOFTWARE FOUNDATION. socket — Low-level networking interface. Disponível em: <https://docs.python.org/3/library/socket.html>. Acesso em: 2026.

[4] PYTHON SOFTWARE FOUNDATION. threading — Thread-based parallelism. Disponível em: <https://docs.python.org/3/library/threading.html>. Acesso em: 2026.

[5] PYTHON SOFTWARE FOUNDATION. sqlite3 — DB-API 2.0 interface for SQLite databases. Disponível em: <https://docs.python.org/3/library/sqlite3.html>. Acesso em: 2026.

[6] RIVERBANK COMPUTING. PyQt6 Reference Guide. Disponível em: <https://www.riverbankcomputing.com/static/Docs/PyQt6/>. Acesso em: 2026.